Kiến Trúc Và Kế Hoạch Triển Khai Hệ Thống Xử Lý Dữ Liệu Giao Dịch Tài Chính Thời Gian Thực (Real-Time Financial Transaction Pipeline)

1. Bối Cảnh Và Phân Tích Bài Toán Kinh Doanh

Trong hệ sinh thái công nghệ tài chính (Fintech) và ngân hàng hiện đại, khối lượng, tốc độ và tính đa dạng của các giao dịch thẻ tín dụng đã đạt đến những mức độ chưa từng có. Việc xử lý hàng ngàn giao dịch mỗi giây không chỉ đòi hỏi một hệ thống lõi (core banking) cực kỳ bền bỉ mà còn tạo ra một lượng dữ liệu khổng lồ cần được phân tích tức thời. Tại các tổ chức tài chính quy mô lớn, một vấn đề nghiêm trọng thường xuyên phát sinh: sự xung đột giữa kiến trúc hệ thống xử lý giao dịch trực tuyến (OLTP - Online Transaction Processing) và nhu cầu phân tích dữ liệu trực tuyến (OLAP - Online Analytical Processing).

Các chuyên gia phân tích dữ liệu (Data Analyst - DA) liên tục phản ánh về những giới hạn của hệ thống hiện tại. Dữ liệu giao dịch thô (raw data) được lưu trữ với dung lượng quá lớn, đồng thời chứa đựng vô số các điểm bất thường như thiếu trường dữ liệu (missing fields), sai định dạng (malformed data), hoặc dữ liệu phi logic như số tiền giao dịch âm. Tình trạng này không chỉ làm sai lệch các báo cáo phân tích mà còn gây ra rủi ro nghiệp vụ nghiêm trọng. Hơn thế nữa, việc các DA thực hiện các truy vấn phân tích phức tạp, các phép kết hợp (JOIN) dữ liệu lớn trực tiếp trên hệ cơ sở dữ liệu lõi của ngân hàng thường dẫn đến hiện tượng khóa bảng (table locks), tiêu tốn tài nguyên CPU và bộ nhớ, từ đó làm chậm hoặc thậm chí gây sập hệ thống giao dịch cốt lõi. Đây là một rủi ro vận hành không thể chấp nhận được trong môi trường tài chính, nơi thời gian gián đoạn (downtime) được tính bằng thiệt hại hàng triệu đô la.

Để giải quyết triệt để bài toán này, hệ thống cần một sự tách ghép (decoupling) hoàn toàn giữa luồng ghi nhận giao dịch và luồng phân tích dữ liệu. Giải pháp chiến lược là xây dựng một Data Pipeline theo kiến trúc Event Streaming sử dụng Apache Kafka làm xương sống.[1] Kiến trúc này sẽ hứng toàn bộ dữ liệu giao dịch theo thời gian thực mà không gây áp lực lên hệ thống lõi. Dữ liệu sau khi được hứng sẽ trải qua quá trình kiểm tra tính hợp lệ (Schema Validation) cực kỳ nghiêm ngặt. Tiếp theo, luồng dữ liệu này sẽ được làm sạch, lọc bỏ các giao dịch lỗi và tổng hợp tự động bằng công nghệ Stream Processing. Cuối cùng, hệ thống phân phối dữ liệu đã được làm sạch và tổng hợp vào một Data Warehouse (PostgreSQL) nhằm phục vụ trực tiếp cho nhu cầu truy vấn và lập báo cáo của các DA. Song song đó, nhằm đáp ứng các yêu cầu về tuân thủ lưu trữ, kiểm toán và phục vụ việc huấn luyện các mô hình học máy (Machine Learning) trong tương lai, toàn bộ dữ liệu thô nguyên bản sẽ được đẩy thẳng vào một Data Lake (MinIO) để lưu trữ dự phòng và không thể thay đổi.

2. Phân Tích Và Lựa Chọn Công Nghệ Nền Tảng

Việc xây dựng một hệ thống có khả năng chịu tải hàng ngàn giao dịch mỗi giây đòi hỏi việc lựa chọn các công nghệ không chỉ tối ưu về mặt hiệu năng mà còn phải đảm bảo tính toàn vẹn và độ tin cậy tuyệt đối của dữ liệu tài chính.

2.1 Định Dạng Dữ Liệu Và Quản Trị Cấu Trúc Bằng Apache Avro

Trong môi trường thực tế của ngành tài chính, việc trao đổi dữ liệu bằng các định dạng văn bản như JSON hay XML bộc lộ nhiều điểm yếu chí mạng, đặc biệt là sự thiếu chặt chẽ về kiểu dữ liệu và kích thước tập tin quá lớn do phải lặp lại tên các trường dữ liệu trong mỗi bản ghi. Thay vào đó, Apache Avro được lựa chọn làm "tiêu chuẩn vàng" cho toàn bộ pipeline này.[2] Avro là một hệ thống tuần tự hóa dữ liệu (data serialization) cung cấp cấu trúc dữ liệu phong phú và lưu trữ dưới định dạng nhị phân cực kỳ nhỏ gọn.[2] Việc sử dụng Avro kết hợp với Confluent Schema Registry mang lại những lợi ích chiến lược:

Thứ nhất, tính nén dữ liệu và hiệu năng. Avro không lưu trữ toàn bộ định nghĩa schema bên trong mỗi tin nhắn. Thay vào đó, nó chỉ lưu trữ một byte định danh (magic byte) và một ID của schema (dài 4 bytes), theo sau là dữ liệu nhị phân thuần túy.[2] Điều này giúp giảm thiểu tối đa băng thông mạng và dung lượng lưu trữ trên Kafka, cho phép thông lượng truyền tải cao hơn gấp nhiều lần so với JSON.

Thứ hai, khả năng tiến hóa cấu trúc (Schema Evolution). Trong quá trình phát triển, cấu trúc dữ liệu giao dịch chắc chắn sẽ thay đổi (ví dụ: thêm trường mã danh mục 가 merchant, hoặc điểm đánh giá rủi ro gian lận). Avro hỗ trợ mạnh mẽ khả năng Schema Evolution, cho phép thêm hoặc bớt các trường dữ liệu mà không làm hỏng (break) các ứng dụng đang sản xuất (Producer) hoặc tiêu thụ (Consumer) dữ liệu.[3] Schema Registry hoạt động như một người gác cổng, đảm bảo rằng mọi thay đổi cấu trúc đều tuân thủ các quy tắc tương thích (ví dụ: Backward Compatibility), từ đó ngăn chặn việc sập hệ thống do lỗi bất đồng bộ dữ liệu.[5]

2.2 Xử Lý Luồng Dữ Liệu Thời Gian Thực Bằng Kafka Streams

Đối với yêu cầu làm sạch và tổng hợp dữ liệu trực tiếp trên luồng sự kiện, Kafka Streams là sự lựa chọn hoàn hảo nhất.[7] Không giống như các framework xử lý dữ liệu lớn khác như Apache Spark hay Apache Flink đòi hỏi một cụm máy chủ (cluster) riêng biệt để vận hành, Kafka Streams là một thư viện Java gốc (native library) của Kafka.[8]

Bằng việc sử dụng Kafka Streams, nhóm kỹ sư có thể triển khai logic xử lý dưới dạng một ứng dụng Java tiêu chuẩn. Ứng dụng này có khả năng mở rộng quy mô một cách linh hoạt bằng cách chạy nhiều phiên bản độc lập; Kafka sẽ tự động phân phối tải công việc thông qua cơ chế phân chia phân vùng (partition assignment). Kafka Streams cung cấp các API cực kỳ mạnh mẽ để thực hiện các phép lọc (filter), kết hợp (join), và đặc biệt là khả năng tổng hợp dữ liệu có trạng thái (stateful aggregation) thông qua cơ chế chia cửa sổ thời gian (windowing).[7] Khả năng xử lý chính xác một lần (exactly-once semantics) của Kafka Streams đảm bảo rằng không có bất kỳ một khoản tiền giao dịch nào bị tính toán trùng lặp hoặc bị bỏ sót, ngay cả trong trường

hợp một tiến trình (node) xử lý bị lỗi và phải khởi động lại.

2.3 Mô Phỏng Dữ Liệu Thực Tế Với Java Producer Và DataFaker

Do đặc thù của ngành tài chính, các API giao dịch nội bộ không bao giờ được phép truy cập công khai để thực hiện các bài kiểm tra hiệu năng. Do đó, hệ thống yêu cầu một công cụ mô phỏng dữ liệu (Simulator) mạnh mẽ để tạo ra từ 1,000 đến 5,000 giao dịch mỗi giây. Một ứng dụng Java Producer sẽ được xây dựng, tích hợp với thư viện DataFaker.

DataFaker là một thư viện mã nguồn mở mạnh mẽ, kế thừa và tối ưu hóa từ JavaFaker, cho phép tạo ra các tập dữ liệu giả lập nhưng có tính logic và mức độ chân thực cực kỳ cao.[9] Thay vì tạo ra các chuỗi ký tự ngẫu nhiên vô nghĩa, DataFaker có khả năng khởi tạo với các bộ định dạng vùng miền (Locale), ví dụ như vi_VN, để sinh ra tên người dùng, địa chỉ và tên cửa hàng mang đậm ngữ cảnh Việt Nam.[9] Các hàm như faker.finance().creditCard() và faker.finance().iban() sẽ được sử dụng để tạo ra các số thẻ tín dụng hợp lệ theo thuật toán Luhn.[12]

Hơn thế nữa, ứng dụng mô phỏng này không chỉ tạo ra dữ liệu theo chiều hướng "happy path" (luôn đúng), mà sẽ được lập trình để phản ánh sự hỗn loạn của môi trường thực tế. Bằng việc sử dụng các hàm phân phối xác suất toán học (ví dụ: phân phối chuẩn - Gaussian distribution), số tiền giao dịch sẽ tập trung ở các giá trị trung bình (ví dụ: 200,000 VNĐ đến 2,000,000 VNĐ) và hiếm khi xuất hiện các giá trị cực đoan. Đồng thời, một tỷ lệ phần trăm nhất định (ví dụ: 2%) các giao dịch sẽ bị cố tình làm lỗi, gán trạng thái thất bại (FAILED), hoặc chứa giá trị số tiền âm (amount < 0) để kiểm chứng khả năng lọc dữ liệu rác của Kafka Streams ở các bước sau.

3. Kiến Trúc Tổng Thể Và Cấu Hình Hạ Tầng (Docker Compose)

Luồng dữ liệu của hệ thống được thiết kế theo một cấu trúc đường ống (pipeline) định tuyến một chiều cực kỳ chặt chẽ, trải qua các giai đoạn từ khi sinh ra cho đến khi được lưu trữ vĩnh viễn.

3.1 Luồng Lưu Chuyển Dữ Liệu (Data Flow Topology)

Hệ thống sẽ chảy theo luồng tuần tự như sau:

1. [Java Producer] liên tục tạo ra các đối tượng giao dịch, chuyển đổi chúng thành định dạng Avro, kết nối với Schema Registry để xác thực, và bắn vào Kafka.
2. `` đóng vai trò là nhật ký sự kiện (event log) hứng toàn bộ các luồng giao dịch thô này.
3. `` tiêu thụ liên tục từ topic thô, loại bỏ các bản ghi không hợp lệ, thực hiện việc cộng dồn số tiền theo từng khoảng thời gian 1 phút cho mỗi cửa hàng, và xuất kết quả ra một topic mới.
4. `` lưu trữ dữ liệu đã được tổng hợp, sạch sẽ và sẵn sàng cho việc phân tích.
5. [Kafka Connect] hoạt động như một hệ thống vận chuyển dữ liệu tự động, sử dụng hai connector song song. Một luồng hút dữ liệu từ topic sạch đẩy vào PostgreSQL. Luồng

còn lại hút dữ liệu từ topic thô đẩy vào MinIO để lưu trữ dự phòng dài hạn.

3.2 Cấu Hình Hạ Tầng Với Docker Compose

Để đảm bảo tính nhất quán và dễ dàng triển khai, toàn bộ hạ tầng cốt lõi sẽ được chứa hóa (containerized) và quản lý bằng Docker Compose. Một yếu tố mang tính đột phá trong kiến trúc này là việc triển khai Apache Kafka ở chế độ KRaft (Kafka Raft Metadata mode).[13] Chế độ KRaft loại bỏ hoàn toàn sự phụ thuộc vào hệ thống Apache ZooKeeper cũ, giúp giảm bớt sự phức tạp của hạ tầng, tiết kiệm tài nguyên bộ nhớ, và tăng tốc độ bầu chọn thủ lĩnh (leader election) cho các phân vùng Kafka khi có sự cố xảy ra.[14]

Bảng dưới đây mô tả chi tiết các dịch vụ (services) sẽ được định nghĩa trong tệp cấu hình docker-compose.yml và vai trò chiến lược của chúng:

Dịch Vụ (Service)	Vai Trò Và Chi Tiết Cấu Hình Kiến Trúc
Kafka (KRaft Mode)	Được định nghĩa với biến môi trường KAFKA_PROCESS_ROLES=broker,controller và KAFKA_ENABLE_KRAFT=yes, cho phép một container duy nhất đảm nhận cả hai vai trò lưu trữ và quản trị cluster.[14] Cổng 9092 được mở ra cho các client bên ngoài, trong khi cổng 29092 được dùng cho giao tiếp mạng nội bộ của Docker.[14]
Schema Registry	Dịch vụ của Confluent, phụ thuộc vào Kafka. Dịch vụ này lưu trữ và phiên bản hóa (versioning) các lược đồ Avro. Nó kết nối với Kafka thông qua cấu hình SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS=PLAINTEXT://kafka:29092 và phơi bày cổng 8081 để Java Producer và Kafka Connect có thể tải các định nghĩa Schema.[15]
MinIO	Hệ thống lưu trữ đối tượng tương thích hoàn toàn với API của Amazon S3. Được cấu hình với tài khoản MINIO_ROOT_USER và MINIO_ROOT_PASSWORD.[16] Một container khởi tạo phụ (init-container) sử dụng công cụ MinIO Client (mc) sẽ được cấu hình để tự động tạo ra bucket lưu trữ tên là

	raw-transactions-backup ngay khi hệ thống khởi động.[16]
PostgreSQL	Cơ sở dữ liệu quan hệ, hoạt động như Data Warehouse phục vụ báo cáo. Cổng 5432 được mở rộng. Cấu hình tự động khởi tạo cơ sở dữ liệu analytics cùng với tài khoản người dùng tương ứng. Cơ sở dữ liệu này được tinh chỉnh để tiếp nhận luồng dữ liệu cập nhật liên tục (upsert) từ Kafka Connect.
Kafka Connect	Khung nền tảng tích hợp dữ liệu. Do hình ảnh Docker mặc định của Kafka Connect không bao gồm sẵn các plugin, kiến trúc yêu cầu viết một Dockerfile tùy chỉnh mở rộng từ confluentinc/cp-kafka-connect. Sử dụng lệnh confluent-hub install để tải và nhúng các gói kafka-connect-jdbc và kafka-connect-s3 vào hệ thống.[17] Nó được cấu hình với URL của Schema Registry để có thể giải mã dữ liệu Avro.[19]

4. Quản Trị Schema Và Thiết Kế Định Dạng Avro

Trong các hệ thống phân tán phức tạp, hợp đồng dữ liệu (data contract) giữa các bên là yếu tố sống còn. Việc thiếu quản trị schema chặt chẽ sẽ dẫn đến tình trạng "rác vào, rác ra" (garbage in, garbage out), làm sập các tiến trình phân tích của Data Analyst.

4.1 Thiết Kế Cấu Trúc Schema Của Giao Dịch

Định dạng Avro yêu cầu một lược đồ được viết bằng định dạng JSON để mô tả chính xác các trường dữ liệu và kiểu dữ liệu tương ứng. Dưới đây là lược đồ (schema) được thiết kế chuyên biệt cho hệ thống giao dịch tài chính này:

JSON

```
{
 "namespace": "com.fintech.pipeline.avro",
 "type": "record",
 "name": "FinancialTransaction",
```

```
  "doc": "Lược đồ mô tả một giao dịch thẻ tín dụng trong thời gian thực",
  "fields":
}
```

Mỗi trường dữ liệu đều được định nghĩa kiểu tĩnh. Trường transactionTimestamp sử dụng kiểu logic timestamp-millis được tích hợp sẵn của Avro, giúp cho các hệ thống hạ nguồn (downstream) tự động hiểu và chuyển đổi thành định dạng ngày giờ (DateTime) thay vì chỉ nhìn nhận nó là một con số nguyên khổng lồ (long).

4.2 Quy Tắc Tiến Hóa Schema (Schema Evolution)

Một nguyên tắc cốt lõi trong vận hành hệ thống tài chính là tính liên tục. Khi nghiệp vụ kinh doanh yêu cầu thêm các trường dữ liệu mới—chẳng hạn như vị trí địa lý của giao dịch (location) hoặc điểm số rủi ro gian lận (fraudScore)—chúng ta không thể dừng toàn bộ hệ thống để cập nhật cấu trúc cơ sở dữ liệu.

Confluent Schema Registry hỗ trợ cấu hình các mức độ tương thích (Compatibility Levels). Đối với dự án này, mức tương thích **BACKWARD** (Tương thích ngược) là cấu hình bắt buộc.[6] Tương thích ngược đảm bảo rằng, các Consumer đang sử dụng phiên bản Schema mới vẫn có khả năng đọc được những dữ liệu cũ do Producer bắn ra với phiên bản Schema cũ.[6] Để tuân thủ nguyên tắc tiến hóa an toàn, bất kỳ một trường dữ liệu nào được bổ sung thêm vào schema sau này đều phải cung cấp một giá trị mặc định (default value).[3] Ví dụ, việc bổ sung thêm trường location phải được cấu trúc với kiểu union-with-null: "type": ["null", "string"], "default": null.[3] Điều này đảm bảo rằng khi Kafka Streams (sử dụng Schema v2) đọc các tin nhắn cũ (được tạo bởi Schema v1 không có trường location), nó sẽ tự động gán giá trị null và không gây ra ngoại lệ (exception) làm sập ứng dụng.

5. Xây Dựng Java Producer Chịu Tải Cao (High-Throughput)

Mục tiêu của Java Producer không chỉ đơn thuần là đẩy dữ liệu vào Kafka, mà phải đạt được thông lượng khổng lồ từ 1,000 đến 5,000 tin nhắn mỗi giây một cách ổn định, đồng thời mô phỏng được các kịch bản lỗi thực tế.

5.1 Kiến Trúc Mô Phỏng Với DataFaker

Trong hàm chính của Producer, thư viện DataFaker được khởi tạo để liên tục sản sinh dữ liệu. Cấu trúc mã lập trình sẽ tạo ra một danh sách cố định gồm 10,000 khách hàng (accountId) và 500 cửa hàng (merchantId) mang tên gọi địa phương tại Việt Nam (thông qua Locale("vi", "VN")).[9] Sự giới hạn số lượng này nhằm tạo ra tính lặp lại của dữ liệu, mô phỏng đúng hành vi mua sắm khi nhiều người dùng giao dịch tại cùng một tập hợp cửa hàng hạn chế.

Trong vòng lặp tạo tin nhắn, hệ thống sẽ sử dụng thuật toán ngẫu nhiên để xác định giá trị cho số tiền (amount). Tuy nhiên, thay vì phân phối đều (uniform distribution) dẫn đến các con số ảo tưởng, hệ thống áp dụng phương pháp phân phối chuẩn (Standard Normal Distribution) thông

qua hàm Random.nextGaussian(). Lập trình viên sẽ điều chỉnh độ lệch chuẩn (Standard Deviation) và giá trị trung bình (Mean) sao cho phần lớn số tiền rơi vào khoảng 50,000 VNĐ đến 2,000,000 VNĐ.

Để cung cấp cơ sở cho việc kiểm thử luồng lọc (filter) ở các bước tiếp theo, một khối logic if/else sẽ được cài cắm: có 2% xác suất giao dịch sẽ bị đảo ngược dấu thành số tiền âm (amount < 0), và 5% xác suất trạng thái giao dịch bị đánh dấu là FAILED. Những dữ liệu nhiễu này là minh chứng rõ ràng cho các phàn nàn của Data Analyst và cần phải được hệ thống pipeline giải quyết.

5.2 Tinh Chỉnh Hiệu Năng Kafka Producer

Mặc định, Kafka Producer được tối ưu hóa cho độ trễ thấp (low latency), nghĩa là nó cố gắng gửi tin nhắn đi ngay lập tức. Tuy nhiên, điều này tạo ra quá nhiều yêu cầu mạng nhỏ lẻ (network requests), làm thắt nút cổ chai băng thông và không thể đạt được con số 5,000 messages/giây.[20] Việc tinh chỉnh các cấu hình (Configurations) là bắt buộc:

Thuộc Tính (Property)	Giá Trị Đề Xuất	Phân Tích Kỹ Thuật Chuyên Sâu
batch.size	131072 (128 KB)	Điều khiển kích thước tối đa của một lô (batch) dữ liệu. Tăng giá trị này lên 128KB giúp Producer gom hàng trăm giao dịch Avro nhỏ gọn vào cùng một gói tin trước khi gửi đi, tối ưu hóa triệt để tài nguyên I/O.[20]
linger.ms	15	Cấu hình này ra lệnh cho Producer tạm chờ tối đa 15 mili-giây để cho phép các tin nhắn khác kịp đến và lấp đầy lô dữ liệu (batch) trước khi đóng gói gửi đi.[20] Sự hy sinh một chút về độ trễ mang lại mức tăng trưởng thông lượng khổng lồ.
compression.type	lz4	Nén lô dữ liệu bằng thuật toán LZ4 trước khi truyền qua mạng. Vì cấu trúc Avro

		vốn dĩ đã nhỏ, việc nén bằng LZ4 tốn rất ít CPU nhưng lại giảm được băng thông truyền tải một cách đáng kể.[20]
acks	all	Đảm bảo rằng tin nhắn gửi đi không bị mất trừ khi toàn bộ cụm cluster sụp đổ. Mặc dù nó làm giảm tốc độ một chút so với acks=1, nhưng đối với hệ thống tài chính, không bao giờ được phép hy sinh tính toàn vẹn dữ liệu để đổi lấy tốc độ.[20]

Bằng cách tích hợp lớp KafkaAvroSerializer của Confluent, toàn bộ dữ liệu này sẽ được gắn một ID tương ứng lấy từ Schema Registry trước khi đẩy vào Kafka topic raw-transactions.[2]

6. Xử Lý Luồng Dữ Liệu Chuyên Sâu Với Kafka Streams

Dữ liệu thô hiện đang cuồn cuộn chảy trong topic raw-transactions. Nhiệm vụ của lớp Kafka Streams là đóng vai trò như một cỗ máy tinh chế, chạy độc lập và xử lý dữ liệu ngay khi nó vừa xuất hiện trong log của Kafka. Việc lập trình Kafka Streams sử dụng các API thiết kế dưới dạng DSL (Domain Specific Language), cung cấp các toán tử hàm (functional operators) mạnh mẽ.

6.1 Lọc Và Làm Sạch Dữ Liệu Bất Thường

Bước đầu tiên trong kịch bản (Topology) của Streams là loại bỏ những dữ liệu gây ra sự phẫn nộ cho đội ngũ Data Analyst. Hệ thống khởi tạo một KStream đọc từ topic raw-transactions. Sau đó, toán tử .filter() được áp dụng trên mỗi bản ghi để kiểm tra các điều kiện nghiệp vụ:

Java

```
KStream<String, FinancialTransaction> cleanStream = rawStream.filter(
    (key, transaction) -> transaction.getAmount() > 0
                    && "SUCCESS".equals(transaction.getStatus())
);
```

Bất kỳ giao dịch nào có số tiền âm, bằng không, hoặc chứa trạng thái thất bại (FAILED) sẽ lập tức bị chặn lại và loại bỏ khỏi luồng. Phương pháp loại trừ từ sớm này giúp giảm tải đáng kể cho

quá trình tính toán và lưu trữ ở các pha tiếp theo.

6.2 Mô Hình Hóa Cửa Sổ Thời Gian (Time-Windowing Aggregation)

Bài toán cốt lõi là "tính tổng số tiền giao dịch theo từng phút của mỗi Merchant". Trong xử lý dữ liệu luồng (stream processing), luồng dữ liệu là vô hạn. Nếu không có khái niệm cửa sổ (Windowing), các hàm tính tổng sẽ tích lũy một con số kéo dài vô tận.[22] Kafka Streams cung cấp các cơ chế chia cửa sổ dữ liệu dựa trên nhãn thời gian (timestamp) của sự kiện.

Có nhiều mô hình chia cửa sổ như: Hopping Windows (cửa sổ nhảy vọt, có sự chồng lấn dữ liệu giữa các cửa sổ) [7], Session Windows (cửa sổ dựa trên phiên hoạt động).[7] Tuy nhiên, để tạo ra các báo cáo thống kê chính xác tuyệt đối theo từng phút mà không có dữ liệu nào bị tính trùng lặp, cơ chế **Tumbling Windows** (Cửa sổ lật) là mô hình duy nhất đáp ứng được yêu cầu.[7] Tumbling Windows tạo ra các khoảng thời gian cố định, liên tiếp và hoàn toàn không giao nhau (non-overlapping).[7] Bất kỳ một giao dịch nào cũng chỉ thuộc về duy nhất một cửa sổ thời gian (ví dụ: từ 10:00:00 đến 10:00:59).

Quá trình tổng hợp được hiện thực hóa qua ba toán tử liên tiếp:

1. .groupByKey(): Nhóm luồng dữ liệu lại theo khóa (key), ở đây giả định khóa là mã cửa hàng (merchantId). Toán tử này ngầm định tạo ra một quá trình chia lại phân vùng (repartition), đảm bảo rằng tất cả các giao dịch của cùng một merchant luôn được định tuyến đến cùng một tiến trình xử lý.[24]
2. .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(1))): Áp dụng cơ chế Tumbling Window với kích thước chính xác là 1 phút.[7]
3. .aggregate(...): Hàm cộng dồn số tiền. Nó đòi hỏi một giá trị khởi tạo (bằng 0.0) và một hàm cập nhật ((key, transaction, total) -> total + transaction.getAmount()).[7] Khối dữ liệu kết quả sẽ được lưu trữ cục bộ trong một cơ sở dữ liệu nhúng tên là RocksDB (được quản lý bởi Materialized.as()) trước khi bắn ra ngoài.[7]

6.3 Kiểm Soát Độ Trễ Bằng Cơ Chế Đè Nén (Suppression)

Một thách thức kinh điển trong kiến trúc xử lý luồng là xử lý dữ liệu đến trễ (late-arriving data) và số lượng bản ghi cập nhật được xuất ra. Theo mặc định, mỗi khi có một giao dịch mới lọt vào cửa sổ 1 phút đang mở, Kafka Streams sẽ lập tức tính toán lại tổng số tiền và gửi một thông báo cập nhật (update) xuống topic đích.[25] Nếu một cửa hàng có 1,000 giao dịch trong vòng 1 phút, Kafka Streams sẽ bắn ra 1,000 thông báo cập nhật liên tục cho cửa sổ đó.[25] Điều này hoàn toàn đi ngược lại mục tiêu giảm tải cho cơ sở dữ liệu, vì PostgreSQL sẽ phải gánh chịu 1,000 thao tác cập nhật (UPDATE) chỉ cho một bản ghi duy nhất.

Giải pháp mang tính quyết định ở đây là áp dụng cơ chế **Suppression** (đè nén). Việc đính kèm toán tử .suppress(Suppressed.untilWindowCloses(unbounded())) vào ngay sau hàm tổng hợp sẽ ra lệnh cho Kafka Streams giữ toàn bộ các kết quả tính toán trung gian ở trong bộ nhớ (hoặc RocksDB).[7] Nó kìm hãm việc phát sóng tin nhắn cho đến khi khoảng thời gian của cửa sổ đó thực sự kết thúc.[25] Kết quả là, thay vì 1,000 lần cập nhật nhiễu loạn, Kafka Streams sẽ chỉ phát

hành một bản ghi duy nhất và chính xác nhất ghi nhận tổng doanh thu của merchant đó cho cả một phút xuống topic clean-transactions.[25]

7. Phân Phối Dữ Liệu Tự Động Bằng Kafka Connect

Sau khi Kafka Streams hoàn thành nhiệm vụ, dữ liệu tổng hợp sạch sẽ đang nằm trong topic clean-transactions, trong khi dữ liệu thô nguyên bản vẫn lưu ở topic raw-transactions. Lớp Kafka Connect đóng vai trò như hệ thống giao vận cuối cùng, hút dữ liệu từ các topic này và chuyển đến đúng đích mà không cần phải viết thêm các ứng dụng tích hợp riêng biệt phức tạp.

7.1 S3 Sink Connector: Lưu Trữ Lịch Sử Dài Hạn Vào MinIO (Data Lake)

Toàn bộ dữ liệu giao dịch thô (raw data) chứa đựng giá trị to lớn cho việc phân tích lịch sử, đối soát giao dịch, huấn luyện các thuật toán phát hiện gian lận và tuân thủ các quy định kiểm toán của nhà nước. Do lượng dữ liệu là quá lớn để duy trì mãi trong Kafka, chúng cần được chuyển sang một hệ thống lưu trữ đối tượng giá rẻ và bền bỉ (Data Lake).

Confluent S3 Sink Connector được sử dụng để trích xuất dữ liệu từ raw-transactions. Dù được thiết kế cho Amazon S3, nó hoàn toàn tương thích và hoạt động xuất sắc với MinIO khi ta ghi đè địa chỉ đích thông qua thuộc tính store.url trỏ về địa chỉ nội bộ của container MinIO (ví dụ: http://minio:9000).[16]

Các cấu hình mang tính chiến lược của S3 Sink Connector bao gồm:

Cấu Hình Connector	Giá Trị	Cơ Chế Hoạt Động Và Mục Đích
format.class	io.confluent.connect.s3.format.avro.AvroFormat	Giữ nguyên định dạng gốc Avro khi ghi xuống MinIO. Điều này đảm bảo dữ liệu luôn đi kèm với cấu trúc schema, tiết kiệm không gian lưu trữ và hoàn toàn tương thích với các công cụ phân tích dữ liệu lớn như Apache Spark hay Presto sau này.[28] Ngoài ra, Parquet cũng là một lựa chọn thay thế rất tốt cho việc truy vấn theo cột.[29]
partitioner.class	TimeBasedPartitioner	Rất quan trọng cho hiệu năng Data Lake. Tính năng

		này sẽ không ném tất cả dữ liệu vào cùng một thư mục. Thay vào đó, nó chia dữ liệu thành các cấu trúc thư mục lồng nhau dựa trên thời gian thực tế (ví dụ: year=2026/month=04/day=05/hour=10/). Điều này cho phép các kỹ sư dữ liệu trong tương lai có thể quét (scan) dữ liệu của một ngày cụ thể mà không cần đọc toàn bộ Data Lake.[27]
flush.size	100000	S3/MinIO hoạt động kém hiệu quả nếu lưu trữ hàng triệu tập tin có dung lượng quá nhỏ (vài KB). Cấu hình này bắt buộc connector phải đệm (buffer) đủ 100,000 bản ghi trước khi nối kết và đẩy thành một tập tin duy nhất xuống MinIO, tối ưu hóa hoạt động đọc/ghi của hệ thống tệp.[27]

7.2 JDBC Sink Connector: Cung Cấp Dữ Liệu Trực Quan Vào PostgreSQL (Data Warehouse)

Điểm kết thúc của hành trình là cung cấp dữ liệu sạch cho các Data Analyst thông qua một cơ sở dữ liệu quan hệ quen thuộc: PostgreSQL. Confluent JDBC Sink Connector chịu trách nhiệm đăng ký (subscribe) vào topic clean-transactions và tự động ánh xạ dữ liệu Avro thành các câu lệnh SQL để cập nhật vào bảng.

Vì kết quả từ Kafka Streams là các cửa sổ thời gian (windowed data), hệ thống có nguy cơ xử lý các sự kiện trùng lặp do việc phục hồi sự cố hoặc do cấu hình sai lệch độ trễ. Nếu Connector chỉ thực thi lệnh INSERT đơn thuần, hệ thống sẽ sớm sụp đổ vì vi phạm lỗi trùng lặp khóa chính (Primary Key Constraint) trong cơ sở dữ liệu PostgreSQL.[30] Do đó, kỹ thuật **Upsert** (Cập nhật hoặc Chèn) là yêu cầu tiên quyết.

Bảng dưới đây trình bày cấu hình cốt lõi để hiện thực hóa cơ chế này một cách an toàn:

Cấu Hình Connector	Giá Trị	Cơ Chế Hoạt Động Và Mục Đích
insert.mode	upsert	Định tuyến hành vi chèn. Connector sẽ phân tích bảng PostgreSQL; nếu bản ghi đã tồn tại (dựa trên khóa chính), nó sẽ thực thi lệnh UPDATE để ghi đè giá trị mới (tổng số tiền). Nếu chưa tồn tại, nó thực thi lệnh INSERT.[30]
pk.mode	record_key	Cực kỳ quan trọng. Cấu hình này chỉ định rằng khóa chính (Primary Key) của bảng PostgreSQL sẽ được trích xuất trực tiếp từ khóa (key) của tin nhắn Kafka.[31] Do Kafka Streams xuất kết quả với khóa là tổ hợp giữa merchantId và thời gian cửa sổ (window timestamp), sự kết hợp này đảm bảo mỗi bản ghi có tính duy nhất tuyệt đối.[31]
auto.create	true	Nếu bảng đích chưa tồn tại trong cơ sở dữ liệu, Connector sẽ đọc lược đồ Avro từ Schema Registry và sử dụng ngôn ngữ DDL để tự động tạo bảng (CREATE TABLE) với các kiểu dữ liệu cột tương ứng.[30]
auto.evolve	true	Kết hợp chặt chẽ với khả năng Schema Evolution đã phân tích ở phần 4. Khi Schema của kết quả có sự bổ sung trường mới, Connector sẽ tự động thực

		thi lệnh ALTER TABLE ADD COLUMN để điều chỉnh cấu trúc bảng mà không cần bất kỳ sự can thiệp thủ công nào.[30]

8. Kế Hoạch Vận Hành Và Giám Sát Hệ Thống

Để đảm bảo hệ thống hoạt động trơn tru trong môi trường sản xuất, việc cấu hình tĩnh là chưa đủ; hệ thống cần được bao bọc bởi các quy trình giám sát và xử lý lỗi chuyên sâu.

Trong Kafka Connect, dù dữ liệu đã được làm sạch, đôi khi những thay đổi đột ngột về kiểu dữ liệu có thể gây ra lỗi khi thực thi lệnh SQL vào PostgreSQL. Để ngăn chặn việc toàn bộ luồng dữ liệu bị tắc nghẽn (pipeline halt) do một bản ghi dị biệt, tính năng **Dead Letter Queue (DLQ)** phải được kích hoạt. Bằng cách thiết lập cấu hình errors.tolerance=all và errors.deadletterqueue.topic.name=postgres-sink-errors, Kafka Connect sẽ tự động bắt giữ các bản ghi gây lỗi, ném chúng vào topic DLQ này, và âm thầm tiếp tục tiến trình xử lý các bản ghi hợp lệ tiếp theo.[18] Data Analyst hoặc kỹ sư hệ thống sau đó có thể phân tích riêng topic DLQ này để tìm hiểu nguyên nhân gốc rễ (root cause analysis).

Hệ thống nên tích hợp xuất các số liệu JMX (Java Management Extensions) ra Prometheus và hiển thị qua Grafana. Các chỉ số sống còn cần được cảnh báo bao gồm:

- Đối với Kafka Streams: Độ trễ xử lý (Processing Latency) và số lượng bản ghi bị bỏ qua (Skipped Records). Sự gia tăng đột biến của Skipped Records chứng tỏ hệ thống sinh dữ liệu đang gặp vấn đề trầm trọng.
- Đối với Kafka Connect: Độ trễ tiêu thụ (Consumer Lag). Nếu Consumer Lag tăng dần theo thời gian, chứng tỏ PostgreSQL đang bị quá tải và không thể theo kịp tốc độ của luồng dữ liệu, đòi hỏi phải tối ưu hóa chỉ mục (indexing) hoặc phần cứng của Database.

Tóm Lược

Bài toán bế tắc về phân tích dữ liệu trực tiếp trên hệ thống cốt lõi đã được giải quyết triệt để thông qua một kiến trúc Event Streaming tách ghép và phân tán hoàn toàn. Bằng việc sử dụng Apache Avro để thiết lập kỷ luật và cấu trúc cho dữ liệu [2], Kafka Streams để tinh chế, lọc bỏ các sự kiện rác và gom nhóm thông qua cơ chế Tumbling Window chặt chẽ [7], hệ thống không những giảm thiểu hàng ngàn lần áp lực ghi (write pressure) lên cơ sở dữ liệu hạ nguồn thông qua sự kiện đè nén (suppression) [25] mà còn mở ra khả năng mở rộng không giới hạn. Dữ liệu sau khi hoàn thiện được phân phối một cách tự động và không có lỗi (fault-tolerant) vào các trung tâm lưu trữ đặc thù bằng Kafka Connect—gửi dữ liệu sạch vào PostgreSQL phục vụ tức thì cho nhu cầu lập báo cáo quản trị, và đổ dữ liệu thô an toàn vào MinIO phục vụ cho vòng đời lưu trữ dài hạn.[18] Kế hoạch kiến trúc và triển khai này thiết lập một chuẩn mực vàng, đảm bảo tổ chức tài chính duy trì được độ tin cậy tuyệt đối, tính sẵn sàng cao, và khả năng thích ứng linh hoạt

trước những thay đổi liên tục của nghiệp vụ trong tương lai.

Nguồn trích dẫn

1. Introduction | Apache Kafka, truy cập vào tháng 4 5, 2026, https://kafka.apache.org/intro/
2. Avro Schema Serializer and Deserializer for Schema Registry on Confluent Platform, truy cập vào tháng 4 5, 2026, https://docs.confluent.io/platform/current/schema-registry/fundamentals/serdes-develop/serdes-avro.html
3. Schema Evolution and Compatibility for Schema Registry on Confluent Platform, truy cập vào tháng 4 5, 2026, https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html
4. Best Practices for Evolving Schemas in Schema Registry - Solace Docs, truy cập vào tháng 4 5, 2026, https://docs.solace.com/Schema-Registry/schema-registry-best-practices.htm
5. Schema Registry For Avro - Meegle, truy cập vào tháng 4 5, 2026, https://www.meegle.com/en_us/topics/schema-registry/schema-registry-for-avro
6. Practical Schema Evolution with Avro | by Elliot West | Expedia Group Technology - Medium, truy cập vào tháng 4 5, 2026, https://medium.com/expedia-group-tech/practical-schema-evolution-with-avro-c07af8ba1725
7. How to Build Windowed Aggregations with Kafka Streams - OneUptime, truy cập vào tháng 4 5, 2026, https://oneuptime.com/blog/post/2026-01-21-kafka-streams-windowed-aggregations/view
8. Kafka Streams Data Types and Serialization for Confluent Platform, truy cập vào tháng 4 5, 2026, https://docs.confluent.io/platform/current/streams/developer-guide/datatypes.html
9. datafaker/README.md at main - GitHub, truy cập vào tháng 4 5, 2026, https://github.com/datafaker-net/datafaker/blob/main/README.md
10. Datafaker 2.0.0 (11 June 2023), truy cập vào tháng 4 5, 2026, https://www.datafaker.net/releases/2.0.0/
11. Basic usage - Datafaker, truy cập vào tháng 4 5, 2026, https://www.datafaker.net/documentation/usage/
12. java-faker/src/main/java/com/github/javafaker/Finance.java at master, truy cập vào tháng 4 5, 2026, https://github.com/DiUS/java-faker/blob/master/src/main/java/com/github/javafaker/Finance.java
13. How to Set Up Apache Kafka in Docker Compose - OneUptime, truy cập vào tháng 4 5, 2026, https://oneuptime.com/blog/post/2026-01-16-docker-kafka/view
14. Kafka KRaft mode with docker-compose - Stack Overflow, truy cập vào tháng 4

5, 2026, https://stackoverflow.com/questions/78807386/kafka-kraft-mode-with-docker-compose
15. Docker Compose for Kafka: single broker in Kraft mode, kafka-ui, confluent rest-proxy, confluent schema-registry | by Tetiana | Medium, truy cập vào tháng 4 5, 2026, https://medium.com/@tetianaokhotnik/docker-compose-for-kafka-single-broker-in-kraft-mode-kafka-ui-and-confluent-rest-proxy-7334da721d4c
16. Kafka 101: Store Kafka Events in MinIO Using Kafka Connect | by Rasheed Shaik - Medium, truy cập vào tháng 4 5, 2026, https://medium.com/@rasheed99/store-kafka-events-in-minio-using-kafka-connect-e19b28bd2ffc
17. blog-assets/kafka/kafka-schema-registry-minio.md at main - GitHub, truy cập vào tháng 4 5, 2026, https://github.com/minio/blog-assets/blob/main/kafka/kafka-schema-registry-minio.md
18. JDBC Sink Connector for Confluent Platform, truy cập vào tháng 4 5, 2026, https://docs.confluent.io/kafka-connectors/jdbc/current/sink-connector/overview.html
19. JDBC Sink Connector (Avro) on Confluent Platform | by Zakir Hossain | Nov, 2023 - Medium, truy cập vào tháng 4 5, 2026, https://zakir-hossain.medium.com/jdbc-sink-connector-avro-on-confluent-platform-locally-ad4b92fa0a40
20. How to Build a High-Throughput Kafka Producer - OneUptime, truy cập vào tháng 4 5, 2026, https://oneuptime.com/blog/post/2026-01-21-kafka-high-throughput-producer/view
21. Optimizing Kafka Producer for High Throughput | by charchit patidar - Medium, truy cập vào tháng 4 5, 2026, https://medium.com/@charchitpatidar/optimizing-kafka-producer-for-high-throughput-f8e808b06bcb
22. Windowing in Kafka Streams - Confluent, truy cập vào tháng 4 5, 2026, https://www.confluent.io/blog/windowing-in-kafka-streams/
23. How to Create Tumbling Windows - OneUptime, truy cập vào tháng 4 5, 2026, https://oneuptime.com/blog/post/2026-01-30-tumbling-windows/view
24. How to aggregate over tumbling windows with Kafka Streams - Confluent Developer, truy cập vào tháng 4 5, 2026, https://developer.confluent.io/confluent-tutorials/tumbling-windows/kstreams/
25. How to emit only the final result of a windowed aggregation in Kafka Streams, truy cập vào tháng 4 5, 2026, https://developer.confluent.io/confluent-tutorials/window-final-result/kstreams/
26. How to output result of windowed aggregation only when window is finished? [duplicate], truy cập vào tháng 4 5, 2026, https://stackoverflow.com/questions/53946013/how-to-output-result-of-windowed-aggregation-only-when-window-is-finished

27. How to Sink Kafka Data to S3 - OneUptime, truy cập vào tháng 4 5, 2026, https://oneuptime.com/blog/post/2026-01-21-kafka-connect-s3-sink/view
28. Journaling Kafka messages with S3 connector and Minio | by Nitish Tiwari - Medium, truy cập vào tháng 4 5, 2026, https://medium.com/cloud-storage/journaling-kafka-messages-with-s3-connector-and-minio-83651a51045d
29. Kafka s3 connector to minio in parquet format - Stack Overflow, truy cập vào tháng 4 5, 2026, https://stackoverflow.com/questions/73532773/kafka-s3-connector-to-minio-in-parquet-format
30. Apache Kafka Guide #62 Kafka Connect JDBC Sink Connector | by Paul Ravvich - Medium, truy cập vào tháng 4 5, 2026, https://medium.com/apache-kafka-from-zero-to-hero/apache-kafka-guide-62-kafka-connect-jdbc-sink-connector-a9840dc24e9c
31. Kafka Connect JDBC Sink deep-dive: Working with Primary Keys - rmoff's random ramblings, truy cập vào tháng 4 5, 2026, https://rmoff.net/2021/03/12/kafka-connect-jdbc-sink-deep-dive-working-with-primary-keys/
32. PostgreSQL (JDBC) Sink Connector for Confluent Cloud Quick Start, truy cập vào tháng 4 5, 2026, https://docs.confluent.io/cloud/current/connectors/cc-postgresql-sink.html
33. Connecting Kafka to MinIO. How to connect data being distributed... | by Alex | The Startup, truy cập vào tháng 4 5, 2026, https://medium.com/swlh/connecting-kafka-to-a-minio-s3-bucket-using-kafka-connect-92d34a81704e